COURSE LABORATORY MANUAL

A. LABORATORY OVERVIEW

Degree:	**B.E.**	Programme:	AI
Semester:	VI	Academic Year:	2025-26
Laboratory Title:	Generative AI	Laboratory Code:	BAIL657C
L-T-P-S:	0-0-2-0	Duration of SEE:	2 Hrs
Total Contact Hours:	24	SEE Marks:	50*
Credits:	1	CIE Marks:	50
Lab Manual Author:	**Mr. Naveena C**		Dt : 27/01/26
Checked By:	**Mr. Venkatesh Y C**		Dt : 27/01/26

*The SEE will be conducted for 100 marks and proportionally reduced to 50 marks.

B. DESCRIPTION

1. PREREQUISITES:

- Programming and Libraries
- Natural Language Processing (NLP)
- Machine Learning Fundamentals
- Deep Learning Foundations
- Generative Models and Architectures
- Neural Network Concepts
- Probability and Statistics (Maths) for AI

2. BASE COURSE:

- Python Programming for Data Science (BDS306B)
- Analysis & Design of Algorithms (BCS401)
- Artificial Intelligence (BAD402)
- Machine Learning (BAI602)
- Natural Language Processing (BAI601)

3. COURSE OUTCOMES:

At the end of the course, the student will be able to;

1. Understand the principles and concepts behind generative AI models.
2. Explain the knowledge gained to implement generative models using Prompt design frameworks.
3. Apply various Generative AI applications for increasing productivity.
4. Develop Large Language Model-based Apps.

4. RESOURSES REQUIRED:

- Jupyter Notebooks
- Google Colab

5. RELEVANCE OF THE COURSE:

- Machine Learning (BAI702)

Prepared by: Mr.Naveena C	Checked by: Mr. Venkatesh Y C	HOD

6. GENERAL INSTRUCTIONS:

- All the students are expected to do the log book entry.
- Be on time and listen to all the instructions given by the faculty / lab instructors.
- Wearing the ID card is compulsory inside the laboratory.
- The observation and record book should be maintained and submitted in each laboratory on time.
- All the students are expected to have a pre-prepared concept which is to be implemented.
- Enter your log out time before leaving the laboratory.
- Do not reconfigure the cabling/equipment or remove anything from the laboratory without prior permission.

7. CONTENTS:

Expt No.	Title of the Experiments	RBT	CO
1	Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.	L3	CO1
2	Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q 1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.	L3	CO1
3	Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.	L3	CO1
4	Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.	L3	CO2
5	Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.	L3	CO3
6	Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.	L3	CO3
7	Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.	L3	CO4
8	Install langchain, cohere (for key), langchain-community. Get the api key (By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.	L3	CO2
9	Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution . How many employees are working in it. A brief 4-line	L3	CO2

	summary of the institution.		
10	Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.	L3	CO4

8. REFERENCE:

1. Modern Generative AI with ChatGPT and OpenAI Models: Leverage the Capabilities of OpenAI's LLM for Productivity and Innovation with GPT3 and GPT4, by Valentina Alto, Packt Publishing Ltd, 2023.
2. Generative AI for Cloud Solutions: Architect modern AI LLMs in secure, scalable, and ethical cloud environments, by Paul Singh, Anurag Karuparti, Packt Publishing Ltd, 2024.

-

C. EVALUATION SCHEME

For CBCS 2022 scheme:

1. Laboratory Components: 30 Marks (Observation Writeup – 5 Marks + Lab Conduction – 10 Marks + Viva-Voce – 5 Marks + Record Writing – 10 Marks).
2. Laboratory IA tests: 20 Marks
   (Minimum 1 IA test is mandatory. IA test shall be conducted for 100 Marks; for the final IA test marks shall be converted to maximum of 20).
3. Continuous Internal Evaluation (CIE) = 30 + 20 = 50 Marks.
4. *SEE* : 50* Marks
   (*The SEE will be conducted for 100 marks and proportionally reduced to 50 marks)

-

D1. ARTICULATION MATRIX

Mapping of CO to PO

	POs											
COs	1	2	3	4	5	6	7	8	9	10	11	12
1. Understand the principles and concepts behind generative AI models.	2	2	2	2	-	-	-	-	-	-	-	-
2. Explain the knowledge gained to implement generative models using Prompt design frameworks.	2	3	3	2	-	-	-	-	-	-	-	-
3. Apply various Generative AI applications for increasing productivity.	2	2	2	-	-	2	-	-	-	-	-	-
4. Develop Large Language Model-based Apps.	3	3	3	3	3	-	-	-	-	-	-	-

Note: Mappings in the Tables D1 (above) and D2 (below) are done by entering in the corresponding cell the Correllation Levels in terms of numbers. For Slight (Low): 1, Moderate (Medium): 2, Substantial (High): 3 and for no correllation: " - ".

D2. ARTICULATION MATRIX CO v/s PSO

Mapping of CO to PSO

	PSOs	
COs	1	2
1. Understand the principles and concepts behind generative AI models.	3	2
2. Explain the knowledge gained to implement generative models using Prompt design frameworks.	3	2
3. Apply various Generative AI applications for increasing productivity.	3	2

4. Develop Large Language Model-based Apps.	3	2

-

E. EXPERIMENTS

1. EXPERIMENT NO: 01

2. TITLE: **Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.**

3. LEARNING OBJECTIVES:

- Understand the concept of word embeddings and how pre-trained models like GloVe represent words as vectors.
- Perform vector arithmetic on word embeddings to discover semantic relationships.
- Analyze and interpret the results of similarity queries and arithmetic operations using word vectors.
- Evaluate the effectiveness of word embeddings in capturing linguistic relationships.

4. AIM:

To explore the properties of pre-trained word embeddings and analyze semantic relationships between words using vector arithmetic techniques.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupytor Notebook

6. THEORY / HYPOTHESIS:

Pre-trained word vectors are representations of words in a multi-dimensional space, typically generated using machine learning algorithms like Word2Vec, GloVe, or fastText. These vectors capture semantic relationships between words, meaning that words with similar meanings are located close together in the vector space.

Popular pre-trained word embedding models include:

Word2Vec: Trained on large text corpora using a neural network.

GloVe: Global Vectors for Word Representation, based on matrix factorization.

FastText: Extends Word2Vec by representing words as n-grams of characters.

Word relationships using vector arithmetic involve operations like addition, subtraction, and averaging of these word vectors to reveal relationships. A classic example is the analogy:

king - man + woman = queen

This is derived by performing vector arithmetic, where the vector for "king" is adjusted by removing the vector for "man" and adding the vector for "woman," resulting in a vector close to "queen."

Paris - France + Italy = Rome

Such operations showcase how word vectors capture not just word meanings, but relationships between words, enabling tasks like analogy solving, clustering, and semantic similarity analysis.

7. PROCEDURE / PROGRAMME / ACTIVITY:

Using Glove model

```
# Install necessary libraries
!pip install gensim numpy
# Import libraries
import gensim.downloader as api
model = api.load("glove-wiki-gigaword-50")
print(model.most_similar(positive=['king', 'woman'], negative=['man']))
# arithmetic operations (e.g., king - man + woman ≈ queen)
result = model.most_similar(positive=['woman', 'king'], negative=['man'])
```

```
print(f"'king' + 'man' - 'woman' is closest to: {result[0]}")
```

Using Word2Vec model

```
# Install necessary libraries
!pip install gensim numpy
# Import libraries
import gensim.downloader as api
import numpy as np
from numpy.linalg import norm
# Load pre-trained word vectors
print("Loading pre-trained word vectors...")
word_vectors = api.load("word2vec-google-news-300")

# Function to perform vector arithmetic and find similar words
def explore_word_relationships(word1, word2, word3):
    try:
        # Get vectors for the input words
        vec1 = word_vectors[word1]
        vec2 = word_vectors[word2]
        vec3 = word_vectors[word3]
        # Perform vector arithmetic: word1 - word2 + word3
        result_vector = vec1 - vec2 + vec3
        # Find the most similar words to the resulting vector
        similar_words = word_vectors.similar_by_vector(result_vector, topn=10)
        # Exclude input words from the results
        filtered_words = [(word, similarity) for word, similarity in similar_words if word not in
{word1, word2, word3}]
        print(f"\nWord Relationship: {word1} - {word2} + {word3}")
        print("Most similar words to the result (excluding input words):")
        for word, similarity in filtered_words[:5]:  # Show top 5 results
            print(f"{word}: {similarity:.4f}")
    except KeyError as e:
        print(f"Error: {e} not found in the vocabulary.")
# Example word relationships to explore
explore_word_relationships("king", "man", "woman")
```

8. RESULTS & CONCLUSIONS:

Using Glove model:

[('queen', 0.8523604273796082), ('throne', 0.7664334177970886), ('prince', 0.7592144012451172), ('daughter', 0.7473883628845215), ('elizabeth', 0.7460219860076904), ('princess', 0.7424570322036743), ('kingdom', 0.7337412238121033), ('monarch', 0.721449077129364), ('eldest', 0.7184861898422241), ('widow', 0.7099431157112122)]

'king' + 'man' - 'woman' is closest to: ('queen', 0.8523604273796082)

Using Word2Vec model:

(Note: what you observed during the execution of program in the lab)

9. LEARNING OUTCOMES :

- Explain how word embeddings capture semantic and syntactic relationships between words.
- Use pre-trained word vector models (e.g., GloVe, Word2Vec) for natural language

processing tasks.
- Demonstrate the ability to perform and interpret vector arithmetic to uncover word relationships.
- Critically assess the strengths and limitations of word embeddings in representing meaning.

10. APPLICATION AREAS:
- Search Engines
- Chatbots & Virtual Assistants
- Machine Translation
- Text Classification
- Information Retrieval

11. REMARKS:

-

1. EXPERIMENT NO: 02

2. TITLE: **Use Dimensionality Reduction (e.g., PCA Or T-SNE) To Visualize Word Embeddings For Q 1. Select 10 Words From A Specific Domain (e.g., Sports, Technology) And Visualize Their Embeddings. Analyze Clusters And Relationships. Generate Contextually Rich Outputs Using Embeddings. Write A Program To Generate 5 Semantically Similar Words For A Given Input.**

3. LEARNING OBJECTIVES:
- Understand the concept of word embeddings and their role in capturing semantic meaning in text.
- Select and apply dimensionality reduction techniques such as PCA and t-SNE to visualize high-dimensional word vectors in 2D space.
- Identify patterns, clusters, and relationships among domain-specific words based on their spatial distribution in the embedding space.
- Implement a program to find and display semantically similar words using pre-trained word embeddings.
- Interpret the contextual relevance of similar words retrieved for a given input word.

4. AIM:
To visualize and analyze semantic relationships between domain-specific words using pre-trained word embeddings and dimensionality reduction techniques, and to generate contextually similar words through embedding-based similarity.

5. MATERIAL / EQUIPMENT REQUIRED:
- Jupytor Notebook

6. THEORY / HYPOTHESIS:

Dimensionality Reduction for Word Embeddings:
Word embeddings are high-dimensional (e.g., 300 dimensions). To visualize them, we reduce their dimensionality to 2D or 3D using:
PCA (Principal Component Analysis): A linear technique that projects data onto orthogonal axes.
t-SNE (t-Distributed Stochastic Neighbor Embedding): A non-linear technique that preserves local relationships between points.
Both PCA (Principal Component Analysis) and t-SNE (t-Distributed Stochastic Neighbor

Embedding) are dimensionality reduction techniques used to visualize high-dimensional data (e.g., word embeddings) in 2D or 3D.
PCA (Principal Component Analysis) is a linear technique that works best with data that has a linear structure. It seeks to identify the underlying principal components in the data by projecting onto lower dimensions, minimizing variance, and preserving large pairwise distances. PCA for global variance: PCA prioritizes capturing the most variance in the data, which can be less useful for visualizing the nuanced relationships between words in an embedding space.
t-SNE (t-distributed Stochastic Neighbor Embedding) is an unsupervised non-linear dimensionality reduction technique for data exploration and visualizing high-dimensional data. Non-linear dimensionality reduction means that the algorithm allows us to separate data that cannot be separated by a straight line. t-SNE excels at preserving local similarities between data points, meaning nearby words in the embedding space will also appear close together in the visualization, making it ideal for understanding semantic relationships between words.
Semantic Similarity: Semantic similarity measures how closely related two words are in meaning. For example: "king" and "queen" are semantically similar. Analogies: "king - man + woman = queen" demonstrates relationships between words.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
import gensim.downloader as api
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import numpy as np

model = api.load("glove-wiki-gigaword-50")

words = ['queen', 'throne', 'prince', 'daughter', 'elizabeth', 'princess', 'kingdom', 'monarch', 'eldest',
'widow']
vectors = np.array([model[word] for word in words])
fig = make_subplots(rows=1, cols=2, subplot_titles=('PCA Visualization', 't-SNE  Visualization'),
horizontal_spacing=0.1)

pca = PCA(n_components=2)
pca_vectors = pca.fit_transform(vectors)

tsne = TSNE(n_components=2, perplexity=5, n_iter=250, random_state=42)
tsne_vectors = tsne.fit_transform(vectors)

fig.add_trace(go.Scatter(x=pca_vectors[:, 0],y=pca_vectors[:, 1],
        mode='markers+text',text=words, textposition="top center",
marker=dict(size=12,color='purple',symbol='diamond', line=dict(color='gold', width=2)),
name='Words'),row=1, col=1)

fig.add_trace(go.Scatter(x=tsne_vectors[:, 0], y=tsne_vectors[:, 1], mode='markers+text',
text=words, textposition="top center", marker=dict( size=12, color='purple',symbol='diamond',
line=dict(color='gold', width=2)),name='Words'),row=1, col=2)
```

```
fig.update_layout(height=600,width=1200,showlegend=False,      template='plotly_white',
title_text="Royal/Monarchy Word Embeddings Visualization", title_x=0.5, font=dict(size=12),
plot_bgcolor='rgba(240,240,255,0.95)',)

fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgray')
fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgray')
fig.show()
```

8. RESULTS & CONCLUSIONS:

Write the result what you have got while executing in lab.

9. LEARNING OUTCOMES :

- Interpret the geometric structure of word embeddings and how they reflect word meaning and relationships.
- Apply dimensionality reduction techniques (PCA, t-SNE) to visualize and analyze semantic clusters in word vector spaces.
- Differentiate semantic relationships based on word proximity and clustering in reduced-dimensional plots.
- Develop a Python program to compute and display semantically similar words for a given input using pre-trained models.
- Evaluate how effectively word embeddings represent contextual and domain-specific semantics.

10. APPLICATION AREAS:

- Intelligent Search and Query Expansion – Enhancing search engines by identifying related terms based on semantics.
- Educational Tools – Supporting language learning or concept exploration through semantic similarity.
- Content Recommendation – Suggesting similar articles, products, or keywords based on contextual embeddings.
- Text Analytics and Summarization – Identifying key terms and relationships for clustering and summarizing large text corpora.
- Semantic Clustering – Grouping similar terms or concepts in applications like topic modeling or ontology creation.
- Conversational AI – Improving chatbot responses by recognizing synonyms and related phrases using vector similarity.

11. REMARKS:

-

1. EXPERIMENT NO: 03

2. TITLE: **Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.**

3. LEARNING OBJECTIVES:

- Understand the architecture and functioning of Word2Vec.
- Preprocess domain-specific textual data for training word embeddings.
- Train a custom Word2Vec model using a small, specialized corpus.
- Evaluate the learned embeddings using similarity tests and clustering.
- Interpret semantic relationships among domain-specific terms captured by the model.

4. AIM:

To train a custom Word2Vec model on a domain-specific corpus (e.g., legal or medical texts) and evaluate how effectively the learned word embeddings capture domain-specific semantics.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Word Embeddings & Word2Vec: Word Embeddings are numerical vector representations of words that capture their meanings and relationships. Word2Vec is a popular neural network-based algorithm that learns word embeddings from a corpus. There are two architectures for Word2Vec:

- CBOW (Continuous Bag of Words): Predicts a word based on surrounding context words.
- Skip-Gram: Predicts surrounding context words given a central word.

Finding Semantically Similar Words: Uses the trained Word2Vec model to retrieve top-N most similar words. Cosine Similarity is used to find similarity between word vectors.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
from gensim.models import Word2Vec
from gensim.utils import simple_preprocess

sentences = [
"This is a legal document about contracts.",
"The court will review the legal case.",
"Medical professionals require specific training.",
"This is a medical report about the patient."]

tokenized_sentences = [simple_preprocess(sentence) for sentence in sentences]
model = Word2Vec(sentences=tokenized_sentences, vector_size=50, window=5, min_count=1,
workers=4, epochs=10)
print(model.wv.most_similar('legal'))
```

8. RESULTS & CONCLUSIONS:

[('case', 0.190416), ('document', 0.045007), ('contracts', -0.010094), ('the', -0.014260), ('report', -0.023167), ('court', -0.043793), ('will', -0.044073), ('review', -0.094199), ('patient', -0.122760), ('require', -0.149906)]

9. LEARNING OUTCOMES :

- Train a Word2Vec model on custom domain-specific corpora using libraries like Gensim.
- Demonstrate understanding of word embeddings and how they encode semantic and syntactic relationships.
- Identify semantic similarities among domain-specific terms based on vector representations.
- Apply domain-trained embeddings in real-world NLP applications like classification, clustering, or information retrieval within that domain.

10. APPLICATION AREAS:

- Domain-specific Chatbots
- Information Retrieval Systems
- Text Clustering and Summarization
- Recommendation Systems

11. REMARKS:

-

1. EXPERIMENT NO: 04

2. TITLE: **Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.**

3. LEARNING OBJECTIVES:

- Understand the role of word embeddings in capturing semantic relationships between words.
- Formulate enriched prompts by expanding original prompts with embedding-based similar terms.
- Interact with a Generative AI model (e.g., GPT) to generate responses based on different prompts.
- Evaluate the impact of prompt engineering using embeddings on the performance of generative models.

4. AIM:

To enhance the quality and relevance of text generated by a Generative AI model by enriching prompts using semantically similar words retrieved from word embeddings, and compare outputs from original vs. enriched prompts.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Word embeddings (GloVe) convert words into vector representations such that similar words have closer vector distances in the embedding space. The model glove-wiki-gigaword-100 provides 100-dimensional word vectors pre-trained on Wikipedia and Gigaword data.

GPT-2 (a Transformer-based model) is a Generative AI model capable of autocompleting text. It predicts the next word in a sequence based on context. The model is pre-trained on large internet-based datasets. It generates text using context from the input prompt.

Prompt Enrichment with Word2Vec: The program replaces a user-defined keyword with its most semantically similar word (found using word embeddings). This helps enhance the prompt and produce a more detailed and meaningful AI-generated response.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install gensim
!pip install transformers
!pip install nltk
```

```
import gensim.downloader as api
from transformers import pipeline
import nltk
import string
from nltk.tokenize import word_tokenize
nltk.download('punkt')
print("Loading pre-trained word vectors...")
word_vectors = api.load("glove-wiki-gigaword-100")
def replace_keyword_in_prompt(prompt, keyword, word_vectors, topn=1):
    """
    Replace only the specified keyword in the prompt with its most similar word.
    Args:
    prompt (str): The original input prompt.
    keyword (str): The word to be replaced with a similar word.
    word_vectors (gensim.models.KeyedVectors): Pre-trained word embeddings.
    topn (int): Number of top similar words to consider (default: 1).
    Returns:
    str: The enriched prompt with the keyword replaced.
    """
    words = word_tokenize(prompt) # Tokenize the prompt into words
    enriched_words = []
    for word in words:
        cleaned_word = word.lower().strip(string.punctuation) # Normalize word
        if cleaned_word == keyword.lower(): # Replace only if it matches the keyword
            try:
                similar_words = word_vectors.most_similar(cleaned_word, topn=topn)
                if similar_words:
                    replacement_word = similar_words[0][0] # Choose the most similar word
                    print(f"Replacing '{word}' → '{replacement_word}'")
                    enriched_words.append(replacement_word)
                    continue # Skip appending the original word
            except KeyError:
                print(f"'{keyword}' not found in the vocabulary. Using original word.")
        enriched_words.append(word) # Keep original if no replacement was made
    enriched_prompt = " ".join(enriched_words)
    print(f"\n Enriched Prompt: {enriched_prompt}")
    return enriched_prompt
print("\nLoading GPT-2 model...")
generator = pipeline("text-generation", model="gpt2")
# Function to generate responses using the Generative AI model
def generate_response(prompt, max_length=100):
    try:
        response = generator(prompt, max_length=max_length, num_return_sequences=1)
        return response[0]['generated_text']
    except Exception as e:
        print(f"Error generating response: {e}")
        return None
original_prompt = "Who is king."
print(f"\n Original Prompt: {original_prompt}")
```

```
# Retrieve similar words for key terms in the prompt
key_term = "king"
# Enrich the original prompt
enriched_prompt = replace_keyword_in_prompt(original_prompt,key_term, word_vectors)
print("\nGenerating response for the original prompt...")
original_response = generate_response(original_prompt)
print("\nOriginal Prompt Response:")
print(original_response)
print("\nGenerating response for the enriched prompt...")
enriched_response = generate_response(enriched_prompt)
print("\nEnriched Prompt Response:")
print(enriched_response)
print("\nComparison of Responses:")
print("\nOriginal Prompt Response Length:", len(original_response))
print("Enriched Prompt Response Length:", len(enriched_response))
print("\nOriginal Prompt Response Detail:", original_response.count("."))
print("Enriched Prompt Response Detail:", enriched_response.count("."))
```

8. RESULTS & CONCLUSIONS:

```
Loading pre-trained word vectors...
Loading GPT-2 model...
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.

Original Prompt: Who is king.
Replacing 'king' → 'prince'

Enriched Prompt: Who is prince .

Generating response for the original prompt...
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.

Original Prompt Response:
Who is king. Does not that make him a tyrant, or a priest or a prophetess."

The next morning's sermon in New York, on an occasion when Christianity was at its weakest in many years, was a great moment. In his address on the New Testament to the New Covenant of Christ the First Presidency, Bishop Pfleger and the Presiding Bishop John McDonough were in a tense state of prayer, and were asked if they had a moment to contemplate the question of

Generating response for the enriched prompt...

Enriched Prompt Response:
Who is prince ... King of all? What's going on?" "Well, this isn't quite what you've been doing." I'd only expected the man to have said, "You're here with a nice little dinner party. Come on, bring a blanket and a dress, and we'll get dressed. And then take one of them to go to the car and take a nap." "That's not quite right, then" -- because the man just had enough. "I

Comparison of Responses:
```

Original Prompt Response Length: 445
Enriched Prompt Response Length: 374
Original Prompt Response Detail: 3
Enriched Prompt Response Detail: 8

(Note what you observed during the prompt enhancement process)

9. LEARNING OUTCOMES :

- Leverage word embeddings (e.g., Word2Vec) to retrieve semantically similar words.
- Enhance prompts using embedding-based word expansion techniques.
- Apply prompt engineering strategies to improve Generative AI model outputs.
- Generate and compare responses using both original and enriched prompts.

10. APPLICATION AREAS:

- Content Generation
- Conversational AI / Chatbots
- AI-Assisted Coding / Documentation
- Search Query Expansion

11. REMARKS:

-

1. EXPERIMENT NO: 05

2. TITLE: **Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.**

3. LEARNING OBJECTIVES:

- Retrieve similar words to a given seed word using a trained Word2Vec model.
- Incorporate similar words into creative writing tasks, such as generating sentences or short paragraphs.
- Design and implement a program that automates the process of seed word expansion and creative text generation.
- Explore the role of semantic expansion in enhancing imagination, creativity, and narrative depth.
- Practice prompt-based storytelling, guided by data-driven vocabulary suggestions.

4. AIM:

To develop a program that takes a seed word, retrieves similar words using word embeddings (e.g., Word2Vec), and uses those words to generate a short, meaningful paragraph for creative writing or storytelling tasks.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Word embeddings are a powerful representation of words in a continuous vector space, allowing them to capture semantic relationships between words. This program leverages pre-trained word embeddings to generate creative sentences and construct a paragraph from a given seed word.

1. Takes a Seed Word → A user provides a single word (e.g., "adventure").
2. Retrieves Similar Words → Finds top N words that are semantically related.
3. Constructs Sentences → Uses these words to form meaningful sentences.
4. Creates a Short Paragraph → Combines the generated sentences into a creative story.

7. PROCEDURE / programme / ACTIVITY:

```
import gensim.downloader as api
import random
import nltk
nltk.download('punkt')
# Load pre-trained word vectors
word_vectors = api.load("glove-wiki-gigaword-100")
print("Word vectors loaded successfully!")

def get_similar_words(seed_word, top_n=5):
    """Retrieve top-N similar words for a given seed word."""
    try:
        similar_words = word_vectors.most_similar(seed_word, topn=top_n)
        return [word[0] for word in similar_words]
    except KeyError:
        print(f"'{seed_word}' not found in vocabulary. Try another word.")
        return []

def generate_sentence(seed_word, similar_words):
    """Create a meaningful sentence using the seed word and its similar words."""
    sentence_templates = [
        f"The {seed_word} was surrounded by {similar_words[0]} and {similar_words[1]}.",
        f"People often associate {seed_word} with {similar_words[2]} and {similar_words[3]}.",
        f"In the land of {seed_word}, {similar_words[4]} was a common sight.",
        f"A story about {seed_word} would be incomplete without {similar_words[1]} and
{similar_words[3]}.",
    ]
    return random.choice(sentence_templates)

def generate_paragraph(seed_word):
    """Construct a creative paragraph using the seed word and similar words."""
    similar_words = get_similar_words(seed_word, top_n=5)
    if not similar_words:
        return "Could not generate a paragraph. Try another seed word."
    paragraph = [generate_sentence(seed_word, similar_words) for _ in range(6)]
    return " ".join(paragraph)

# Example usage
seed_word = input("Enter a seed word: ")
paragraph = generate_paragraph(seed_word)
print("\nGenerated Paragraph:\n")
print(paragraph)
```

8. RESULTS & CONCLUSIONS:
Loading pre-trained word vectors...
Word vectors loaded successfully!
Enter a seed word: travel
Generated Paragraph:
A story about travel would be incomplete without trips and trip. The travel was surrounded by travelers and trips. A story about travel would be incomplete without trips and trip. The travel was surrounded by travelers and trips. In the land of travel, flights was a common sight. In the land of travel, flights was a common sight.

9. LEARNING OUTCOMES :
- Use word embeddings to find semantically similar words to a given seed word.
- Apply embedding-based vocabulary expansion to support creative language generation.
- Enhance narrative development by introducing meaningful, thematically connected words.
- Generate coherent and imaginative content using NLP techniques.
- Demonstrate creative writing skills enhanced by AI tools.

10. APPLICATION AREAS:
- Educational Tools
- Game Development
- Marketing and Advertising
- Content Creation for Social Media
- Language Learning Applications

11. REMARKS:

-

1. EXPERIMENT NO: 06

2. TITLE: **Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.**

3. LEARNING OBJECTIVES:
- Understand the concept of sentiment analysis and its role in natural language processing (NLP).
- Load and use a pre-trained NLP model from the Hugging Face Transformers library.
- Apply the sentiment analysis pipeline to classify the emotional tone (positive, negative, or neutral) of text.
- Simulate real-world applications such as analyzing customer reviews, tweets, or feedback using AI models.
- Explore practical use of transformer-based models in applied AI tasks with minimal setup and coding effort.

4. AIM:
To utilize a pre-trained sentiment analysis model from the Hugging Face Transformers library to classify the sentiment of given text inputs, simulating a real-world application scenario such as product reviews, customer feedback, or social media monitoring.

5. MATERIAL / EQUIPMENT REQUIRED:
- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Sentiment analysis is a Natural Language Processing (NLP) technique used to determine the emotional tone of a given text. It classifies text as positive, negative, or neutral.

Example:"This product is amazing!"→Positive, "I'm very disappointed with the service."→ Negative, "It was an average experience."→ Neutral

Transformers and Pre-trained Models: A powerful deep-learning-based NLP framework by Hugging Face, it uses a pre-trained model for sentiment analysis.

Pipeline in Hugging Face: The pipeline("sentiment-analysis") function loads a pre-trained model and applies it directly to text input. The underlying model is typically DistilBERT, BERT, or RoBERTa, which are transformer-based architectures designed for NLP tasks.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
# Install required libraries
!pip install transformers
# Import the sentiment analysis pipeline from Hugging Face
from transformers import pipeline
# Load the sentiment analysis pipeline
print(" Loading Sentiment Analysis Model...")
sentiment_analyzer = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english", framework="pt”)

def analyze_sentiment(text):
    """
    Analyze the sentiment of a given text input.
    Args:
        text (str): Input sentence or paragraph.
    Returns:
        dict: Sentiment label and confidence score.
    """
    result = sentiment_analyzer(text)[0]  # Get the first result
    label = result['label']  # Sentiment label (POSITIVE/NEGATIVE)
    score = result['score']  # Confidence score
    # Print the input text with sentiment analysis results
    print(f"\n Input Text: {text}")
    print(f" Sentiment: {label} (Confidence: {score:.4f})\n")
    return result

# Example real-world application: Customer feedback analysis
customer_reviews = [
    "The product is amazing! I love it so much.",
    "I'm very disappointed. The service was terrible.",
    "It was an average experience, nothing special.",
    "Absolutely fantastic quality! Highly recommended.",
    "Not great, but not the worst either."
]
# Analyze sentiment for multiple reviews
print("\n Customer Sentiment Analysis Results:")
for review in customer_reviews:
```

```
analyze_sentiment(review)
```

8. RESULTS & CONCLUSIONS:

```
Loading Sentiment Analysis Model...
Customer Sentiment Analysis Results:
 Input Text: The product is amazing! I love it so much.
 Sentiment: POSITIVE (Confidence: 0.9999)
 Input Text: I'm very disappointed. The service was terrible.
 Sentiment: NEGATIVE (Confidence: 0.9998)
 Input Text: It was an average experience, nothing special.
 Sentiment: NEGATIVE (Confidence: 0.9995)
 Input Text: Absolutely fantastic quality! Highly recommended.
 Sentiment: POSITIVE (Confidence: 0.9999)
 Input Text: Not great, but not the worst either.
 Sentiment: NEGATIVE (Confidence: 0.9961)
```

9. LEARNING OUTCOMES :

- Perform sentiment analysis using a pre-trained transformer model from the Hugging Face library.
- Classify the sentiment of text inputs (e.g., positive, negative, neutral) in real-world contexts such as reviews or social media.
- Interpret prediction results, including sentiment labels and confidence scores.
- Understand the practical application of transformer models in customer experience, business intelligence, and social media analysis.

10. APPLICATION AREAS:

- Customer Reviews Analysis
- Social Media Monitoring
- Feedback Analysis
- Market Research

11. REMARKS:

-

1. EXPERIMENT NO: 07

2. TITLE: **Summarize long texts using a pre-trained summarization model using Hugging Face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.**

3. LEARNING OBJECTIVES:

- Understand the concept of text summarization and its importance in NLP.
- Set up and use the Hugging Face's Transformers library and Hugging Face text summarization pipeline, to get efficient text summarization.
- Apply pre-trained models for text summarization tasks, including choosing an appropriate model and fine-tuning parameters.
- Evaluate the quality of generated summaries based on their conciseness, coherence, and retention of key information.

4. AIM:

To perform text summarization using a pre-trained model from Hugging Face's Transformers library, implementing a text summarization pipeline to generate concise summaries of long passages.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text. Summarize long texts using a pre-trained Hugging Face model. It utilizes the summarization pipeline to take a passage as input and generate a concise summary.
Here's a Python program that runs in Jupyter Notebook to summarize long texts using a pre-trained Hugging Face model. It utilizes the summarization pipeline to take a passage as input and generate a concise summary.

7. PROCEDURE / programme / ACTIVITY:

```
# Install required libraries (only needed for first-time setup)
!pip install transformers
# Import the summarization pipeline from Hugging Face
from transformers import pipeline
# Load a more accurate pre-trained summarization model
print("Loading Summarization Model (BART)...")
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

# Function to summarize text with improved accuracy
def summarize_text(text, max_length=None, min_length=None):
  # Remove unnecessary line breaks
  text = " ".join(text.split())

  # Auto-adjust summary length based on text size
  if not max_length:
    max_length = min(len(text) // 3, 150)
  if not min_length:
    min_length = max(30, max_length // 3)

# Generate the summary with different approaches
  summary_1 = summarizer(text, max_length=150, min_length=30, do_sample=False)
  summary_2 = summarizer(text, max_length=150, min_length=30, do_sample=True,
temperature=0.9)
  summary_3 = summarizer(text, max_length=150, min_length=30, do_sample=False,
num_beams=5)
  summary_4 = summarizer(text, max_length=150, min_length=30, do_sample=True, top_k=50,
top_p=0.95)

# Print summarized outputs
  print("\nOriginal Text:")
  print(text)
  print("\nSummarized Text:")
  print("\nDefault:", summary_1[0]['summary_text'])
```

```
  print("\nHigh randomness:", summary_2[0]['summary_text'])
  print("\nConservative:", summary_3[0]['summary_text'])
  print("\nDiverse sampling:", summary_4[0]['summary_text'])

# Example long text passage
long_text = """
Artificial Intelligence (AI) is a rapidly evolving field of computer science focused on creating intelligent machines capable of mimicking human cognitive functions such as learning, problem-solving, and decision-making. In recent years, AI has significantly impacted various industries, including healthcare, finance, education, and entertainment. AI-powered applications, such as chatbots, self-driving cars, and recommendation systems, have transformed the way we interact with technology. Machine learning and deep learning, subsets of AI, enable systems to learn from data and improve over time without explicit programming. However, AI also poses ethical challenges, such as bias in decision-making and concerns over job displacement. As AI technology continues to advance, it is crucial to balance innovation with ethical considerations to ensure its responsible development and deployment.
"""
# Summarize the passage
summarize_text(long_text)
```

8. RESULTS & CONCLUSIONS:

The py code helped to summarize all the content and show in suitable manner.

9. LEARNING OUTCOMES :

- Understand how to use pre-trained summarization models from Hugging Face for text processing.
- Efficiently load and utilize the Hugging Face pipeline for text summarization in Python.
- Gain experience in using state-of-the-art NLP models for text summarization tasks.

10. APPLICATION AREAS:

- Summarizing news articles, research papers, and legal documents.
- Creating executive summaries for long reports.
- Enhancing customer support by condensing feedback.

11. REMARKS:

-

1. EXPERIMENT NO: 08

2. TITLE: **Install langchain, cohere (for key), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive. Create a prompt template to display the output in a particular manner.**

3. LEARNING OBJECTIVES:

- Learn how to use LangChain for prompt engineering.
- Explore Cohere API Integration for text generation.
- Design Prompt Templates Effectively

4. AIM:

to automate the extraction of learning objectives and aims from educational text documents using

LangChain, Cohere API, and Google Drive integration in a modular Python workflow.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

Cohere is an AI-powered Natural Language Processing (NLP) platform that provides large language models (LLMs) for text generation, classification, summarization, search, and embedding. It is similar to OpenAI's GPT models but focuses on enterprise-level AI applications, offering scalable APIs for developers.

Cohere provides different LLMs for various tasks:

- Command – Best for instruction-following and structured responses.
- Generate – Used for creative writing and text expansion.
- Embed – Converts text into vector embeddings for semantic search.

LangChain is an open-source framework designed to simplify the development of applications using Large Language Models (LLMs). It enables easy integration with different LLM providers like Cohere, OpenAI, and others, and helps in chaining multiple AI-driven components like prompt templates, memory, and agents for building complex applications.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
# Step 1: Install required libraries (Run once)
!pip install langchain cohere langchain-community

# Step 2: Import necessary libraries
import cohere
import getpass
from langchain.prompts import PromptTemplate
from langchain_community.llms import Cohere

import getpass
# Step 3: Load the Text File from Local Disk
file_path = "a.txt"  # Update this path as needed
try:
    with open(file_path, "r", encoding="utf-8") as file:
        text_content = file.read()
        print(" File loaded successfully!")
except Exception as e:
    print(" Error loading file:", str(e))

# Step 4: Set Up Cohere API Key
COHERE_API_KEY = getpass.getpass(" Enter your Cohere API Key: ")

# Step 5: Initialize Cohere Model with LangChain
cohere_llm = Cohere(cohere_api_key=COHERE_API_KEY, model="command")
print("API key received?" , bool(COHERE_API_KEY))  # Should print: True

# Step 6: Create a Prompt Template
template = """
You are an AI assistant helping to summarize and analyze a text document.
Here is the document content:
```

```
{text}
□ □Summary:
- Provide a concise summary of the document.
□ □Key Takeaways:
- List 3 important points from the text.
□ □Sentiment Analysis:
- Determine if the sentiment of the document is Positive, Negative, or Neutral.
"""
prompt_template = PromptTemplate(input_variables=["text"], template=template)

#  Step 7: Format the Prompt and Generate Output
formatted_prompt = prompt_template.format(text=text_content)
response = cohere_llm.predict(formatted_prompt)

# Step 8: Display the Generated Output
print("\n **Formatted Output** ")
print(response)
```

8. RESULTS & CONCLUSIONS:
a.text file content extrcted and expressed as template given format.

9. LEARNING OUTCOMES :

- Successfully integrate and use LangChain with Cohere API.
- Design and utilize prompt templates for structured text analysis.
- Write clean, modular, and reusable Python code for text processing tasks.
- Gain hands-on experience with API authentication and secure key management.

10. APPLICATION AREAS:

- Academic Research: Automatically summarizing and analyzing academic papers, reports, or articles.
- Business Intelligence: Quickly extracting insights and sentiment from meeting notes or business documents.
- Content Review

11. REMARKS:

-

1. EXPERIMENT NO: 09

2. TITLE: **Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution. How many employees are working in it. A brief 4-line summary of the institution.**

3. LEARNING OBJECTIVES:

- Understand how to use the Pydantic library in Python for data validation and schema definition.
- Gain practical experience in integrating Wikipedia API with Python for automated information retrieval.

- Understand how to design and implement a customized query chain using the LangChain framework.
- Explore methods for extracting specific information (e.g., founder, year of establishment, branches, employees, and a summary) from unstructured text data.

4. AIM:

To build an automated system that retrieves specific information about an institution from Wikipedia using a custom Pydantic schema and a custom output parser.

5. MATERIAL / EQUIPMENT REQUIRED:

- Jupyter Notebook

6. THEORY / HYPOTHESIS:

This Python program integrates various libraries and concepts to extract institution-related data from Wikipedia and structure it using Pydantic. Provides an interactive interface using ipywidgets.

Libraries and Their Roles:

- wikipedia-api: A Python library used to interact with Wikipedia and fetch pages. Provides access to the Wikipedia REST API to extract article text and metadata.
- pydantic: A data validation and settings management library using Python type annotations. BaseModel is used to define a schema for institution details, ensuring structured data.
- ipywidgets: Used to create interactive widgets (text boxes, buttons) in Jupyter Notebooks. Provides a user-friendly interface for input and output.
- IPython.display: Enables rich output display in Jupyter Notebooks, allowing dynamic updates of results.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install wikipedia-api
from pydantic import BaseModel
import wikipediaapi
from typing import List, Optional

# Define the InstitutionDetails model
class InstitutionDetails(BaseModel):
  founder: Optional[str]
  founded: Optional[str]
  branches: Optional[List[str]]
  number_of_employees: Optional[int]
  summary: Optional[str]

# Function to fetch institution details from Wikipedia
def fetch_institution_details(name: str) -> InstitutionDetails:
  wiki_wiki = wikipediaapi.Wikipedia(user_agent='your-user-agent', language='en')
# Corrected language code
  page = wiki_wiki.page(name)
  if not page.exists():
    raise ValueError(f"'{name}' does not exist on Wikipedia.")
  info = {line.split(':')[0].strip(): line.split(':')[-1].strip() for line in page.text.split('\n') if ':' in line}
  return InstitutionDetails(founder=info.get('Founder', None), founded=info.get('Founded', None),
      branches=info.get('Branches', '').split(',') if 'Branches' in info else None,
      number_of_employees=int(info.get('Number of employees', '0').replace(',', '') or 0),
      summary=page.summary[:500])
```

```
# Function to display institution details
def display_institution_details(details: InstitutionDetails):
  print(f"Founder: {details.founder or 'N/A'}\nFounded: {details.founded or 'N/A'}\n"
        f"Branches: {', '.join(details.branches) if details.branches else 'N/A'}\n"
        f"Employees: {details.number_of_employees or 'N/A'}\n"
        f"Summary: {details.summary or 'N/A'}")

# Main program logic
def main():
  institution_name = input("Enter institution name: ").strip()
  if institution_name:
    try:
    details = fetch_institution_details(institution_name)
    display_institution_details(details)
   except ValueError as e:
    print(e)
  else:
    print("❌Please enter a valid institution name.")
if __name__ == "__main__":
   main()
```

8. RESULTS & CONCLUSIONS:

Institution: MIT
Fetch Details
Founder: N/A
Founded: N/A
Branches: N/A
Employees: N/A
Summary: The Massachusetts Institute of Technology (MIT) is a private research university in Cambridge, Massachusetts, United States. Established in 1861, MIT has played a significant role in the development of many areas of modern technology and science.
In response to the increasing industrialization of the United States, William Barton Rogers organized a school in Boston to create "useful knowledge." Initially funded by a federal land grant, the institute adopted a polytechnic model that stressed

9. LEARNING OUTCOMES :

- Define and enforce data schemas using Pydantic for reliable and structured data parsing.
- Implement custom output parsers to convert unstructured text responses into structured, validated data formats.
- Integrate external knowledge sources such as Wikipedia API into a data processing pipeline.
- Construct and execute chained queries using LangChain to automate information retrieval tasks.
- Extract key factual information (e.g., founder, foundation date, branches, employee count, summary) from natural language sources efficiently.

10. APPLICATION AREAS:

- Automated Knowledge Retrieval Systems
- Information Extraction

- Data Validation and Processing
- Legal, Educational, and Research Tools
- Chatbots and Virtual Assistants
- Content Summarization and Reporting

11. REMARKS:

-

1. EXPERIMENT NO: 10

2. TITLE: **Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.**

3. LEARNING OBJECTIVES:
- Implement natural language processing (NLP) techniques for text analysis.
- Design and build a domain-specific chatbot using large language models (LLMs).
- Create effective prompts for accurate and context-aware chatbot responses.
- Understand and process legal documents effectively (Indian Penal Code).

4. AIM:
To create a chatbot that accurately answers user questions about the Indian Penal Code.

5. MATERIAL / EQUIPMENT REQUIRED:
- Jupyter Notebook

6. THEORY / HYPOTHESIS:

This Python program creates an interactive chatbot focused on answering questions related to the Indian Penal Code (IPC). It utilizes AI language models, Wikipedia data extraction, and data validation techniques to provide accurate, structured legal information.
Libraries and Their Roles:
- LangChain: A framework for developing applications powered by language models. It provides utilities for building prompts, managing LLM (Large Language Model) chains, and integrating different AI models seamlessly.
- Cohere: A platform offering state-of-the-art NLP models for text generation, classification, and more. Here, it powers the AI responses using its "command" model, which is optimized for complex text-to-text tasks like answering legal queries.
- Pydantic: A data validation and parsing library that enforces data models using Python type hints. It ensures that the chatbot's responses are structured, reliable, and adhere to defined schemas.
- Wikipedia-API: A Python library that interacts with Wikipedia to fetch information. It enables the chatbot to pull the Indian Penal Code content directly from Wikipedia.
- IPython Widgets (ipywidgets): A library that allows for the creation of interactive widgets in Jupyter Notebooks, making it possible to build user-friendly interfaces.

A chatbot is an artificial intelligence (AI) application designed to simulate human-like conversations with users through text or voice interactions. Chatbots can be broadly classified into two types:
- **Rule-based Chatbots:** These operate using predefined rules and scripts.
- **AI-Powered Chatbots:** These leverage natural language processing (NLP) and machine learning (ML) techniques to understand and respond to user queries intelligently.

Cohere in LangChain Framework: Cohere is an LLM provider integrated into LangChain, a framework for building intelligent applications. Cohere's models can be used to generate text, answer questions, and provide explanations based on the IPC content. The model is fine-tuned to generate legally accurate responses by using context-specific prompts.

Prompt engineering is the technique of designing the right input prompts to guide LLMs in generating accurate and contextually appropriate responses.

7. PROCEDURE / PROGRAMME / ACTIVITY:

```
!pip install langchain cohere langchain_community wikipedia-api pydantic ipywidgets
# Step 1: Import Libraries
import getpass
from typing import Optional
from langchain import PromptTemplate
from langchain_community.llms import Cohere  # Updated import for LangChain community
models
from pydantic import BaseModel
import wikipediaapi
from IPython.display import display
import ipywidgets as widgets

# Step 2: Enter Cohere API Key
COHERE_API_KEY = getpass.getpass('Enter your Cohere API Key: ')
cohere_llm = Cohere(cohere_api_key=COHERE_API_KEY, model="command")

# Step 3: Fetch IPC Content from Wikipedia
def fetch_ipc_summary():
    user_agent = "IPCChatbot/1.0 (contact: myemail@example.com)"
    wiki_wiki = wikipediaapi.Wikipedia(user_agent=user_agent, language='en')
    page = wiki_wiki.page("Indian Penal Code")
    if not page.exists():
        raise ValueError("The Indian Penal Code page does not exist on Wikipedia.")
    return page.text[:5000]  # Limit to first 5000 characters
ipc_content = fetch_ipc_summary()

# Step 4: Define Response Schema
class IPCResponse(BaseModel):
    section: Optional[str]
    explanation: Optional[str]

# Step 5: Define Prompt Template
prompt_template = PromptTemplate(input_variables=["ipc_content", "question"],
    template="""
You are a legal assistant chatbot specialized in the Indian Penal Code (IPC).
Refer to the following IPC document content to answer the user's question.
{ipc_content}
User Question: {question}
Provide a detailed answer, mentioning the relevant section if applicable.
""")
```

```
# Step 6: Get Chatbot Response
def get_ipc_response(question: str) -> IPCResponse:
    formatted_prompt = prompt_template.format(ipc_content=ipc_content, question=question)
    response = cohere_llm.predict(formatted_prompt)
    # Simple logic to extract a section number if present
    if "Section" in response:
        section_start = response.find("Section")
        section = response[section_start:].split(":")[0].strip()
        explanation = response.split(":", 1)[-1].strip()
    else:
        section = None
        explanation = response.strip()
    return IPCResponse(section=section, explanation=explanation)

# Step 7: Create Jupyter Widgets for Interaction
def display_response(response: IPCResponse):
    print(f" Section: {response.section if response.section else 'N/A'}")
    print(f" Explanation:\n{response.explanation}")

def on_button_click(b):
    user_question = text_box.value
    try:
        response = get_ipc_response(user_question)
        display_response(response)
    except Exception as e:
        print(f" Error: {e}")
# Create input box and button
text_box = widgets.Text(value='', placeholder='Ask about the Indian Penal Code...',
    description='You:', disabled=False)
button = widgets.Button(description='Ask', button_style='info',
    tooltip='Click to ask a question about IPC', icon='gavel')
button.on_click(on_button_click)
# Show widgets
display(text_box, button)
```

8. RESULTS & CONCLUSIONS:

Enter your Cohere API Key: ·····

You: Explain me about section 370

Ask

Section: Section 370 of the Indian Penal Code addresses the penalty for theft.
Explanation: Whoever, guilty of theft, intentionally converts, for his own use, the property stolen from any other description to his own use, or the use of any person other than the true owner, or intentionally deals with it in any manner dishonestly prejudicial to the owner's interest, shall, in addition to the punishment provided for the theft, be liable to a further imprisonment of one year, or a fine, or both. If the value of the thing stolen exceeds five hundred rupees, it shall be liable to a further imprisonment for a term which may extend to two years, or a fine, or both. In the case of repetition of the offence within a period of one year, the offender shall be liable to a punishment for

a term not less than—quadruple the punishment provided for such theft, in addition to the punishment provided for the first offence; or two years' imprisonment, in addition to the ensuing punishment for the second offence, and so on, in addition to the punishment for the third and every subsequent offence.

9. LEARNING OUTCOMES :

- Understand and Utilize Large Language Models (LLMs).
- Know how to work with Prompt Engineering.
- Understand API Integration in Python.
- Create Interactive Widgets
- Design a Domain-Specific Chatbot

10. APPLICATION AREAS:

- Chatbot system
- Prompt Engineering for Legal Queries
- API integration

11. REMARKS: